```
using System.Net.Http.Headers;
using System.IO;
using System.Runtime.CompilerServices;
namespace WheelOfFortune
{
  internal class Program
  {

    //userChoice in main menu
    public static int userChoice;



    //struct student
    public struct Student
    {
      public string firstName;
      public string lastName;
      public string interest;
      public int score;
    }

    //create streamwriter and streamreader
    public static StreamWriter sw;
    public static StreamReader sr;

    public static void Main(string[] args)
    {
      //initializing variables
```

```
    Student[] student = new Student[23];
    Student[] finalists = new Student[10];
    Student selectedPlayer = new Student();

    //call the text file
    LoadStudents(student);

    //runs the program while userChoice is not the exit code
    do
    {
      selectedPlayer = Output(student, finalists, selectedPlayer);
    } while (userChoice != 0);
}

//get the students from the text file to avoid dependencies of each task
public static void LoadStudents(Student[] student)
{
    int i = 0;
    sr = new StreamReader(@"../../../wheelOfFortune.txt");
    while(!sr.EndOfStream && i < student.Length)
    {
      student[i].firstName = sr.ReadLine();
      student[i].lastName = sr.ReadLine();
      student[i].interest = sr.ReadLine();
      student[i].score = int.Parse(sr.ReadLine());
      i++;
    }
    sr.Close();
```

```
}

//taskone to display students ascending by last name
public static void TaskOne(Student[] student, Student selectedPlayer)
{
   Console.Write("First Name".PadRight(25));
   Console.Write("Last Name".PadRight(25));
   Console.Write("Interest".PadRight(50));
   Console.WriteLine("Score".PadRight(25));
   Console.WriteLine();

   //display students
   for(int i = 0; i < student.Length; i++)
   {
      for(int j = 0; j < i; j++)
      {
         //bubble sorting strings
         if (student[j].lastName.CompareTo(student[i].lastName) > 0)
         {
            Student temp = student[i];
            student[i] = student[j];
            student[j] = temp;
         }
      }

   }

   for(int i = 0; i < student.Length; i++)
```

```
  {
    Console.Write($"{student[i].firstName}".PadRight(25));
    Console.Write($"{student[i].lastName}".PadRight(25));
    Console.Write($"{student[i].interest}".PadRight(50));
    Console.WriteLine($"{student[i].score}".PadRight(25));
  }
}

//task to find a specific student by last name
public static void TaskTwo(Student[] student)
{
  int choice;
  do
  {
    Console.Clear();

    //created a mini menu if the user wants to update a student interest
    Console.WriteLine("1 - Find a student by last name");
    Console.WriteLine("0 - Go back");
    Console.Write("Enter your choice: ");
    choice = int.Parse(Console.ReadLine());
    switch (choice)
    {
      case 1:
        //clears the screen for UI purposes
        Console.Clear();
        int studentFoundNumber = 0;
        bool studentFound = false;
```

```
                do
                {

                  Console.Write("Enter a student last name: ");
                  string lastNameSearch = Console.ReadLine();
                  for (int i = 0; i < student.Length; i++)
                  {
                    //matched student name and userinput in lowercase accurately
                    if (lastNameSearch.ToLower() == student[i].lastName.ToLower())
                    {
                      studentFound = true;
                      studentFoundNumber = i;
                    }
                  }

                  //condition if student was find and update their interest
                  if (studentFound == true)
                  {
                    Console.WriteLine("Student found!");
                    Console.WriteLine($"{student[studentFoundNumber].firstName}
{student[studentFoundNumber].lastName}");
                    Console.Write("Update the interest: ");
                    string interest = Console.ReadLine();
                    student[studentFoundNumber].interest = interest;
                    sw = new StreamWriter(@"../../../wheelOfFortune.txt");
                    Console.WriteLine("List updated! Check the text file.");
                    for (int i = 0; i < student.Length; i++)
                    {
```

```
                sw.WriteLine(student[i].firstName);
                sw.WriteLine(student[i].lastName);
                sw.WriteLine(student[i].interest);
                sw.WriteLine(student[i].score);
            }
            sw.Close();
            Console.ReadLine();
          }
          else
          {
            Console.WriteLine("Student not found! Try again!");
          }
        } while (studentFound == false);


        break;
      case 0:
        Console.WriteLine("Returning to main menu.");
        return;
      default:
        Console.WriteLine("Invalid input! Please try again!");
        Console.ReadLine();
        break;
    }
  } while (choice != 0);

}
```

```
//method to generate 10 student finalist to play the wheel of fortune
public static void TaskThree(Student[] student, Student[] finalists)
{
   //initialize variables
   Random rand = new Random();
   int[] arrayRandom = new int[finalists.Length];
   int randomGenerator;

   Console.WriteLine("Generating 10 finalists:");
   Console.WriteLine();
   Console.Write("First Name".PadRight(25));
   Console.Write("Last Name".PadRight(25));
   Console.Write("Interest".PadRight(50));
   Console.WriteLine("Score".PadRight(25));
   Console.WriteLine();

   //outer loop to generate 10 students
   for (int i = 0; i < finalists.Length; i++)
   {
      //random generator to find the student according to the struct
      randomGenerator = rand.Next(0, student.Length-1);

      //inner loop that will start at 0 if a name has been duplicated
      for(int k = 0; k < i; k++)
      {
         //condition if duplication occurs
         while (arrayRandom[k] == randomGenerator)
         {
```

```
            randomGenerator = rand.Next(0, student.Length);
            //k will be assigned 0 and the the loop again to find if duplication happened
            k = 0;
          }
        }

        arrayRandom[i] = randomGenerator;
        finalists[i] = student[randomGenerator];
    }

    //display 10 finalists
    for(int i = 0; i < finalists.Length; i++)
    {
        Console.Write($"{finalists[i].firstName}".PadRight(25));
        Console.Write($"{finalists[i].lastName}".PadRight(25));
        Console.Write($"{finalists[i].interest}".PadRight(50));
        Console.WriteLine($"{finalists[i].score}".PadRight(25));
    }
}

//method to select one student among the 10 finalists
//this method will not work if finalists are not generated
public static Student TaskFour(Student[] finalists, Student selectedPlayer)
{
    Random rand = new Random();
    int randomGenerator;
    randomGenerator = rand.Next(0, finalists.Length);
    Console.WriteLine("Congratulations! You have selected");
```

```
        selectedPlayer = finalists[randomGenerator];
        Console.WriteLine($"{selectedPlayer.firstName}{selectedPlayer.lastName}
{selectedPlayer.score}");
        Console.WriteLine("Press enter to go back.");
        return selectedPlayer;
    }


    //this method will determine the specific spin value once generated by the random
    public static int SpinWheel()
    {
        //initialize variables and create an instance of Random()
        Random rand = new Random();
        int wheelGenerator;
        int[] wheelOfFortune = { 300, 350, 400, 500, 750, 900, 1000, 1500, 1750, 2000,
2250, 2500, 2750, 3000, 3250, 3500, 3700, 4000, 4250, 4500, 4750, 5000 , -2000, -3000
};
        wheelGenerator = rand.Next(0, wheelOfFortune.Length);

        //simple animation to display the current spinvalue
        for (int i = 0; i < 12; i++)
        {
            int displayGenerator = rand.Next(0, wheelOfFortune.Length);

            Console.Write($"\rSpinning... {wheelOfFortune[displayGenerator],6}");
            Thread.Sleep(100 + (i * 20));
        }
```

```
        //this output provides the accurate value of the spin.
        Console.Write($"\rSpinning... {wheelOfFortune[wheelGenerator],6}");
        Thread.Sleep(500);
        Console.WriteLine();
        return wheelOfFortune[wheelGenerator];
    }

    //method to return a string value based on the randomized string
    public static string WordRandomizer()
    {
        Random rand = new Random();
        int wordRandomizer;
        string[] words = { "computerized", "electricity", "chrysanthemum", "programming",
"mathematics", "photography", "dictionary", "restaurant"};
        wordRandomizer = rand.Next(0, words.Length);
        return words[wordRandomizer];
    }

    //a method to check if userInput is a letter from the alphabet
    public static string ValidateUserInput(string userInput, char[] alphabet)
    {
        bool letterFound = false;
        //asks the user to keep entering a string
        do
        {
            //userInput is greated than 1 or no userInput at all
            if (userInput.Length != 1)
            {
```

```
            Console.WriteLine("Invalid input! Try again!");
            Console.Write("Enter a letter: ");
            userInput = Console.ReadLine();
        }

        else
        {
            //checks if userInput is equal to the specific letter in the alphabet
            for (int i = 0; i < alphabet.Length; i++)
            {
                if (char.Parse(userInput.ToUpper()) == alphabet[i])
                {
                    letterFound = true;
                }
            }
            //condition if userInput is not a letter
            if(letterFound == false)
            {
                Console.WriteLine("Invalid input! Try again!");
                Console.Write("Enter a letter: ");
                userInput = Console.ReadLine();
            }
        }
    } while (letterFound == false);
    return userInput;
}

//method to check if a letter is in the word to be guessed.
```

```
    public static int ValidateCorrectLetters(string wordToGuess, char[] wordBlanks,
string userInput)
    {
        //initialize variables
        bool correctLetter = false;
        char validator = char.Parse(userInput);
        int letterOccurences = 0;

        //for loop to check letter occurences
        for(int i = 0; i < wordBlanks.Length; i++)
        {
            if(char.ToLower(wordToGuess[i]) == char.ToLower(validator))
            {
                correctLetter = true;
                wordBlanks[i] = validator;
                letterOccurences++;
            }

        }

        //condition if letter is not in the wordGuess
        if(correctLetter == false)
        {
            Console.WriteLine("Letter not found!");
        }
        else
        {
            Console.WriteLine("Letter found!");
```

```
    }

    //returns the number of letter occurences
    return letterOccurences;
}



//a method created to validate if userInput is a vowel
public static bool IsVowel(string userInput)
{
    userInput = userInput.ToLower();
    if(userInput == "a"||
        userInput == "e"||
        userInput == "i"||
        userInput == "o"||
        userInput == "u")
    {
        return true;
    }
    return false;
}



//the spin wheel method
public static Student TaskFive(Student selectedPlayer)
{
    //initialize variables
    int wheel = 0, currentScore = selectedPlayer.score, letterOccurences = 0;
```

```
string wordToGuess = "", userInput = "", updatedWord = "";
char[] alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ".ToCharArray();

//a constant for a vowel amount
const int VOWEL_REDUCE = 250;
bool isVowel = false;

//get the word from WordRandomizer method
wordToGuess = WordRandomizer();

//create blanks based on the wordGuess length
char[] wordBlanks = new char[wordToGuess.Length];
for (int i = 0; i < wordToGuess.Length; i++)
{
   wordBlanks[i] = '_';
}

//spin the wheel
do
{
   Console.Clear();
   //display the blanks
   for (int i = 0; i < wordBlanks.Length; i++)
   {
      Console.Write($"{wordBlanks[i]} ");
   }
   Console.WriteLine();
   //display the available letters
```

```
for(int i = 0; i < alphabet.Length; i++)
{
   Console.Write($"{alphabet[i]} ");
}



Console.WriteLine("\n");
wheel = SpinWheel();
Console.WriteLine("\n\n");
Console.WriteLine($"The wheel landed on: {wheel}");

Console.WriteLine("\n");
Console.WriteLine($"Current score: {currentScore}");

//validate the input if it is a vowel and score is less than 250
do
{
   Console.Write("Enter a letter: ");
   userInput = Console.ReadLine();
   userInput = ValidateUserInput(userInput, alphabet);
   isVowel = IsVowel(userInput);
   if(isVowel == true && currentScore < VOWEL_REDUCE)
   {
      Console.WriteLine("Can't afford a vowel yet!");
   }
} while (isVowel == true && currentScore < VOWEL_REDUCE);

//removes a letter that was already used
```

```
        for(int i = 0; i < alphabet.Length; i++)
        {
           if (alphabet[i] == char.Parse(userInput.ToUpper()))
           {
              alphabet[i] = ' ';
           }
        }

        //calls the function to see if letters have occured
        letterOccurences = ValidateCorrectLetters(wordToGuess, wordBlanks,
userInput);



        //condition to check the spinvalue
        if (wheel == -2000 || wheel == -3000)
        {
           currentScore += wheel;
        }
        else if(isVowel == true)
        {
           currentScore -= VOWEL_REDUCE;
        }
        else
        {
           currentScore += letterOccurences * wheel;
        }

        //parses wordBlanks to string and assigns to updatedWord
```

```
//to display if the word has been solved
updatedWord = new string(wordBlanks);
Console.ReadLine();
//final display
Console.Clear();

//gives the illusion of resetting the used letters, if the problem has been solved
for (int i = 0; i < wordBlanks.Length; i++)
{
   Console.Write($"{wordBlanks[i]} ");
}
Console.WriteLine();

for (int i = 0; i < alphabet.Length; i++)
{
   Console.Write($"{alphabet[i]} ");
}
Console.WriteLine();

Console.WriteLine($"\nCurrent score: {currentScore}");

//if else condition to check if updatedWord to lower is equal to
//wordToGuess to lower
if (updatedWord.ToLower() == wordToGuess.ToLower())
{
   Console.WriteLine("You have guessed the word!");
   Console.WriteLine($"The word is: {wordToGuess}");
   Console.WriteLine("Press enter to view the top 5");
```

```
            Console.ReadLine();
        }

        //spins the wheel again
        else
        {
            Console.WriteLine("Press enter to spin again!");
            Console.ReadLine();
        }
    } while (updatedWord.ToLower() != wordToGuess.ToLower());

    //sets the current score to the player
    selectedPlayer.score = currentScore;
    return selectedPlayer;
}


//method to update the text file
public static void UpdateScore(Student[] student,Student selectedPlayer)
{
    sw = new StreamWriter(@"../../../wheelOfFortune.txt");
    //update current score
    for(int i = 0; i < student.Length; i++)
    {
        if (student[i].firstName == selectedPlayer.firstName && student[i].lastName ==
selectedPlayer.lastName)
        {
            student[i].score = selectedPlayer.score;
```

```
    }
  }

  //write in text file
  for(int i = 0; i < student.Length; i++)
  {
    sw.WriteLine(student[i].firstName);
    sw.WriteLine(student[i].lastName);
    sw.WriteLine(student[i].interest);
    sw.WriteLine(student[i].score);
  }
  sw.Close();
}



//displays the top five players
public static void TopFive(Student[] student)
{
  //used bubble sort to order players according to their scores
  for(int i = 0; i < student.Length; i++)
  {
    for(int j = 0; j < i; j++)
    {
      if (student[i].score > student[j].score)
      {
        Student temp = student[i];
        student[i] = student[j];
        student[j] = temp;
```

```
        }
      }
    }

    //neatly output top 5 players
    Console.WriteLine("Top 5:\n");
    Console.Write("First Name".PadRight(25));
    Console.Write("Last Name".PadRight(25));
    Console.Write("Interest".PadRight(50));
    Console.WriteLine("Score".PadRight(25));
    Console.WriteLine();
    for (int i = 0; i < 5; i++)
    {
      Console.Write($"{student[i].firstName}".PadRight(25));
      Console.Write($"{student[i].lastName}".PadRight(25));
      Console.Write($"{student[i].interest}".PadRight(50));
      Console.WriteLine($"{student[i].score}".PadRight(25));
    }
    Console.WriteLine("Press enter to go back to main menu.");
}

//method to exit program
public static void TaskZero()
{
    Console.WriteLine("Exiting program. Thank you and have a good day!\n");
}

//error message in the main menu
```

```
    public static void ErrorMessage()
    {
        Console.WriteLine("Wrong input! Please try again!\n");
    }



    //main menu UI
    public static void TaskChoices()
    {
        Console.Clear();
        Console.SetCursorPosition(0, 0);
        Console.WriteLine("1 - List all contestants");
        Console.WriteLine("2 - Locate a player");
        Console.WriteLine("3 - Generate 10 finalists");
        Console.WriteLine("4 - Select the player from the finalists");
        Console.WriteLine("5 - Play the game");
        Console.WriteLine("0 - Exit the game");
        Console.Write("Please Enter a task number: ");
        userChoice = int.Parse(Console.ReadLine());
        Console.Clear();
        Console.SetCursorPosition(0, 0);
    }


    //main menu method that accepts struct and returns the selected student
    public static Student Output(Student[] student, Student[] finalists, Student
selectedPlayer)
    {
        //display main menu UI
```

```
TaskChoices();


//selects a specific task
switch (userChoice)
{
   case 1:
      //display all students
      TaskOne(student, selectedPlayer);
      Console.WriteLine();
      break;
   case 2:
      //select a specific student
      TaskTwo(student);
      break;
   case 3:
      //generate finalists
      TaskThree(student, finalists);
      break;
   case 4:
      //generate a player
      //condition if a finalists is null
      if (string.IsNullOrWhiteSpace(finalists[0].firstName))
      {
         Console.WriteLine("You need to generate finalists first!");
         Console.WriteLine("Press enter to go back.");
      }
      else
      {
```

```
            selectedPlayer = TaskFour(finalists, selectedPlayer);
        }
        break;
    case 5:
        //play the game
        //condition if selectedPlayer is null
        if(string.IsNullOrWhiteSpace(selectedPlayer.lastName))
        {
            Console.WriteLine("You need to find a player first!");
            Console.WriteLine("Press enter to go back.");
        }
        else
        {
            //selected player plays the game
            selectedPlayer = TaskFive(selectedPlayer);

            //updates the score and writes it in the text file
            UpdateScore(student, selectedPlayer);
            Console.Clear();

            //displays top 5 as soon as the game is finished
            TopFive(student);
        }
        break;
    case 0:
        //exits the system
        TaskZero();
        break;
```

```
            default:
                //userinput is wrong
                ErrorMessage();
                break;
        }
        Console.ReadLine();
        return selectedPlayer;
    }
  }
}
```